

Practical 3

API Integration and Data Rendering in React

GitHub REST API with useEffect, Async State Management, Loading/Error/Success States, Search Filter & Retry

Student Name:	Mantra Patel
Student ID:	24DCS076
Course:	Advanced Web Development Frameworks (ITUE301)
Institute:	DEPSTAR - CHARUSAT
Branch / Sem:	B.Tech CSE - Semester 5
Academic Year:	2025-2026
Dev Server Port:	http://localhost:5175

Objective: Integrate the live GitHub REST API into the portfolio Projects page using useEffect, manage async request lifecycle states, and render dynamic repository data with search filtering and error recovery.

1. Theory & Concepts

Practical 3 introduces asynchronous data fetching into a React application using the Fetch API and the `useEffect` lifecycle hook. Data from a REST endpoint is modelled as a three-state machine:

- * Loading - request is in-flight; display a spinner.
- * Success - data received; display the repository grid.
- * Error - request failed; display an error message with a Retry action.

Separation of concerns is enforced by extracting all HTTP communication into a dedicated service module (`src/services/githubApi.js`), keeping `Projects.jsx` purely as a UI orchestrator.

- `useEffect(fn, [dep])`: Schedules a side-effect after every render where `dep` changes. Empty `[]` runs only on mount.
- `useCallback(fn, [dep])`: Memoises the `loadRepositories` function so `useEffect`'s dependency array is stable and never causes an infinite loop.
- GitHub REST API: GET `https://api.github.com/users/Mantra-17/repos?sort=updated&per_page=30`
- Response fields used: `name`, `html_url`, `description`, `language`, `stargazers_count`, `forks_count`, `updated_at`.
- HTTP error classification in `githubApi.js`: 404 -> user not found, 403 -> rate limit, 5xx -> server error.
- Search filter: repos filtered by name, description, and language on every `searchTerm` keystroke.
- Empty state: `RepoList` renders a friendly 'No repositories found' card when `filteredRepos.length === 0`.
- Demo toggle: simulates a network error on demand so the error state and retry can be demonstrated during viva.

2. Component & Service Architecture

Practical 3 - Module Architecture

```
src/
  services/
    githubApi.js      <- isolated HTTP module; throws typed errors
  components/
    Spinner.jsx       <- animated ring; role=status for accessibility
    ErrorMessage.jsx  <- error text + Retry callback button
    RepoCard.jsx      <- single repository card (name, URL, stars, lang)
    RepoList.jsx      <- grid of RepoCards + empty state
  pages/
    Projects.jsx      <- state machine: loading/error/success + search
```

3. Key Code Snippets

Projects.jsx - State Machine with `useEffect`

```
// Projects.jsx - mandatory state + useEffect
```

```

const [repos, setRepos] = useState([])
const [loading, setLoading] = useState(true)
const [error, setError] = useState(null)
const [searchTerm, setSearchTerm] = useState('')

const loadRepositories = useCallback(async () => {
  setLoading(true); setError(null)
  try {
    const data = await fetchUserRepos('Mantra-17')
    setRepos(data)
  } catch (err) {
    setError(err.message)
  } finally {
    setLoading(false)
  }
}, [simulateError])

useEffect(() => { loadRepositories() }, [loadRepositories])

```

githubApi.js - Isolated API Service Layer

```

// githubApi.js - dedicated API service
const BASE_URL = 'https://api.github.com/users'

export async function fetchUserRepos(username = 'Mantra-17') {
  const res = await fetch(`${BASE_URL}/${username}/repos?sort=updated&per_page=30`)
  if (!res.ok) {
    if (res.status === 404) throw new Error(`User not found: ${username}`)
    if (res.status === 403) throw new Error('Rate limit exceeded - wait and retry.')
    throw new Error(`HTTP ${res.status}: ${res.statusText}`)
  }
  return res.json()
}

```

Conditional UI State Rendering

```

// Conditional rendering in Projects.jsx
{loading && <Spinner message='Fetching live repositories...' />}

{!loading && error && (
  <ErrorMessage message={error} onRetry={loadRepositories} />
)}

{!loading && !error && (
  <RepoList repos={filteredRepos} searchTerm={searchTerm} />
)}

```

Real-time Repository Search Filter

```
// Search / Filter
const filteredRepos = repos.filter(repo =>
  repo.name.toLowerCase().includes(searchTerm.toLowerCase()) ||
  (repo.description && repo.description.toLowerCase().includes(searchTerm.toLowerCase())) ||
  (repo.language && repo.language.toLowerCase().includes(searchTerm.toLowerCase()))
)
```

4. Application Screenshots

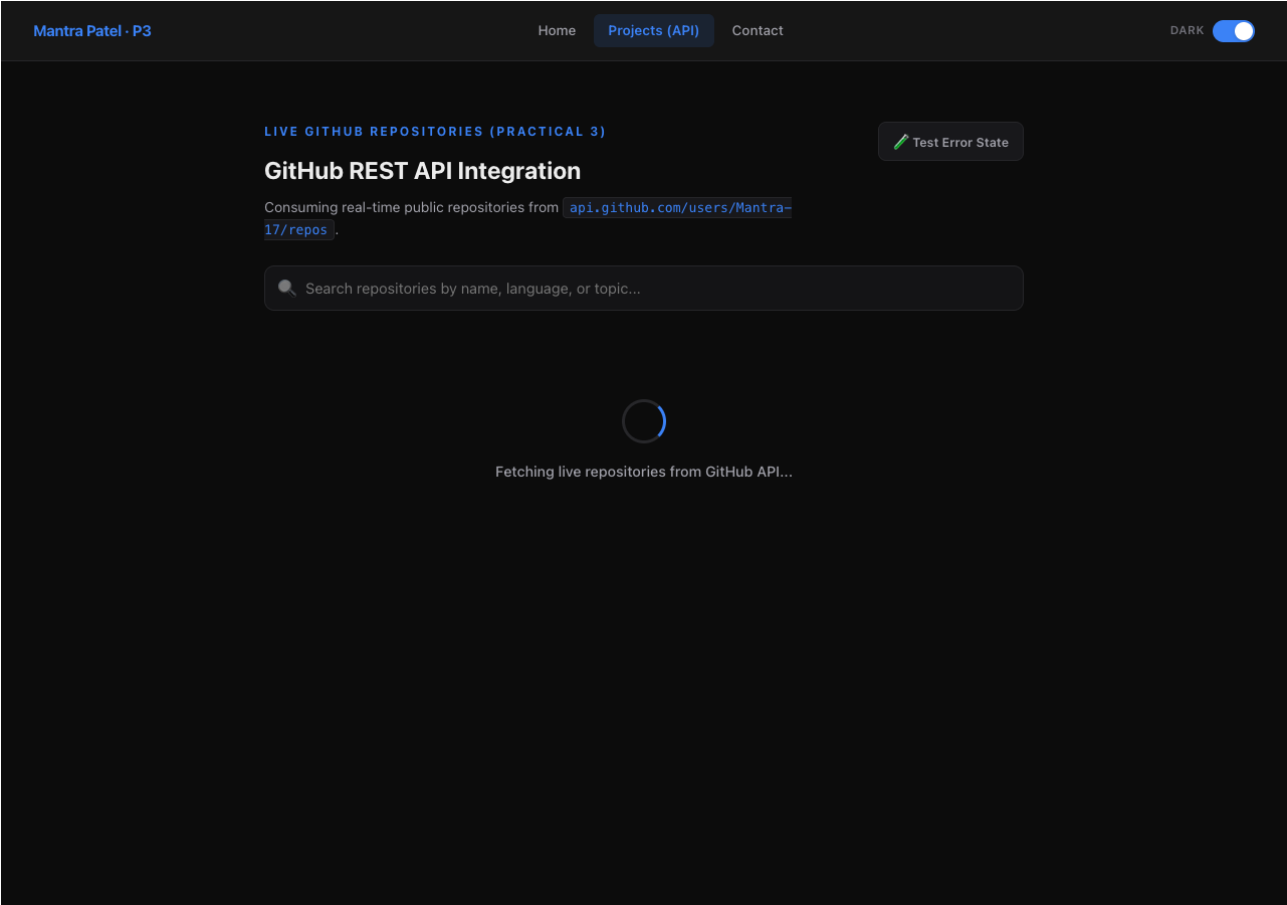


Figure: Practical 3 - Loading State: animated spinner while GitHub API request is in flight

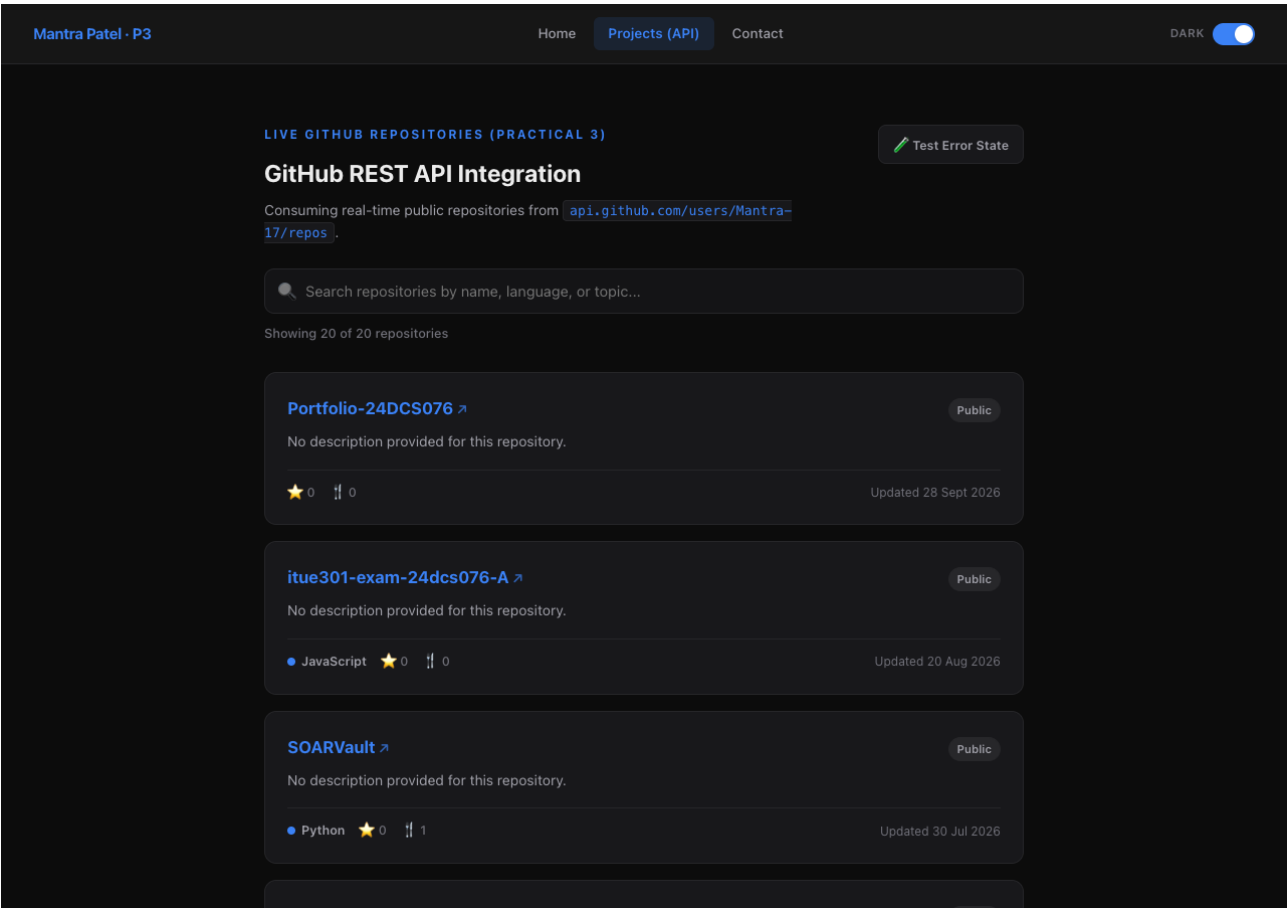


Figure: Practical 3 - Success State: live repository cards from `api.github.com/users/Mantra-17/repos`

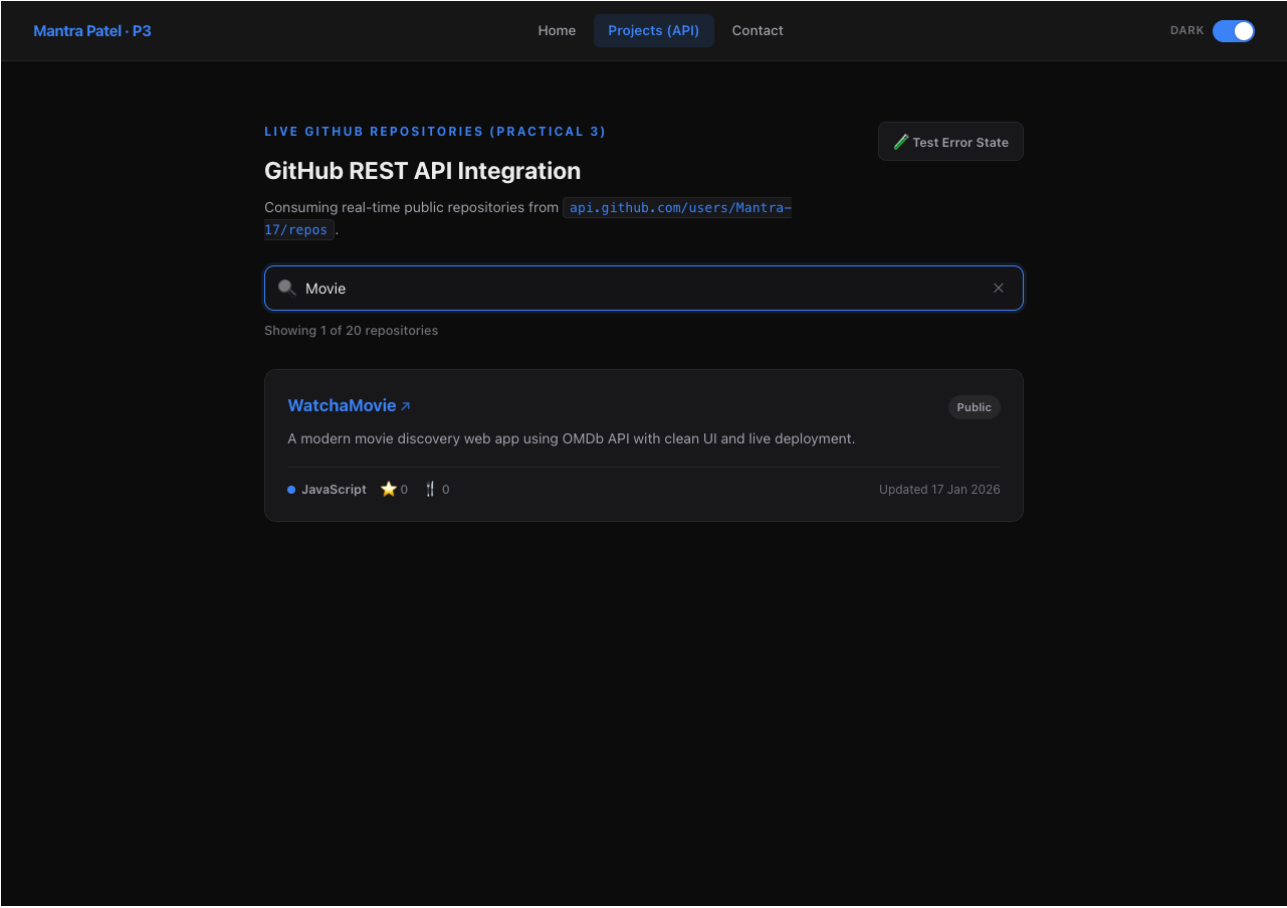


Figure: Practical 3 - Search Filter: 'Movie' keyword narrows results in real-time

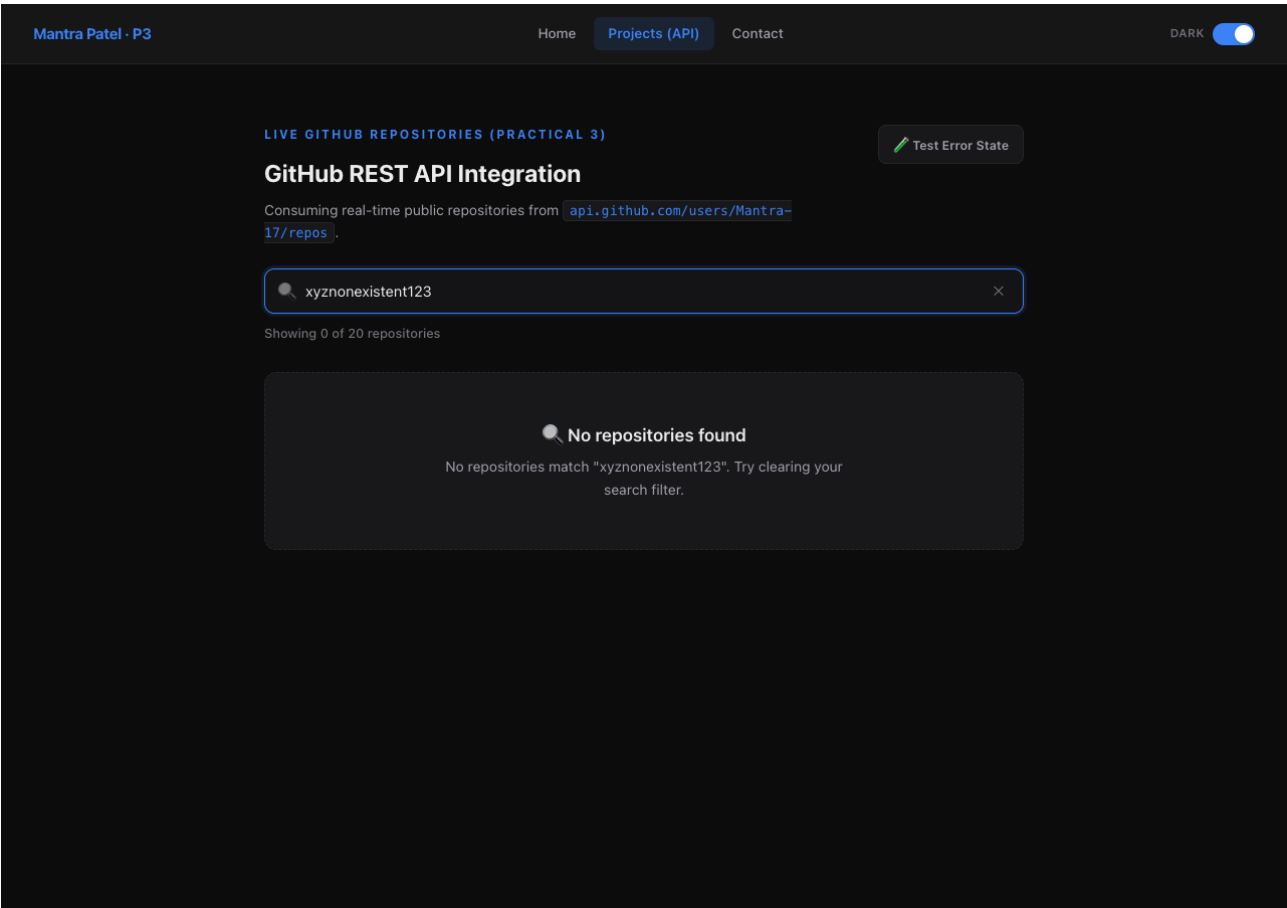


Figure: Practical 3 - Empty State: no repos match the search term 'xyznonexistent123'

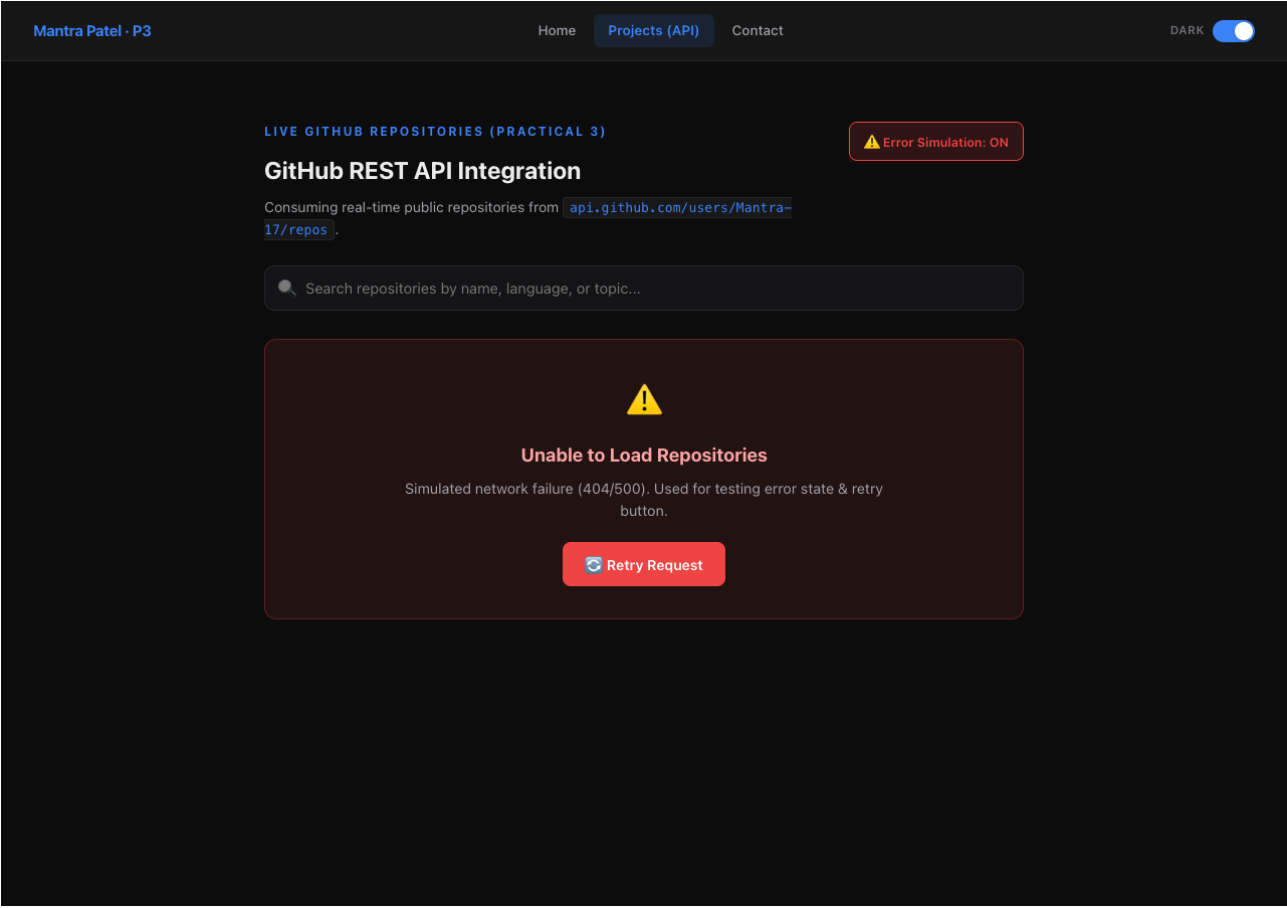


Figure: Practical 3 - Error State: simulated failure showing ErrorMessage + Retry button

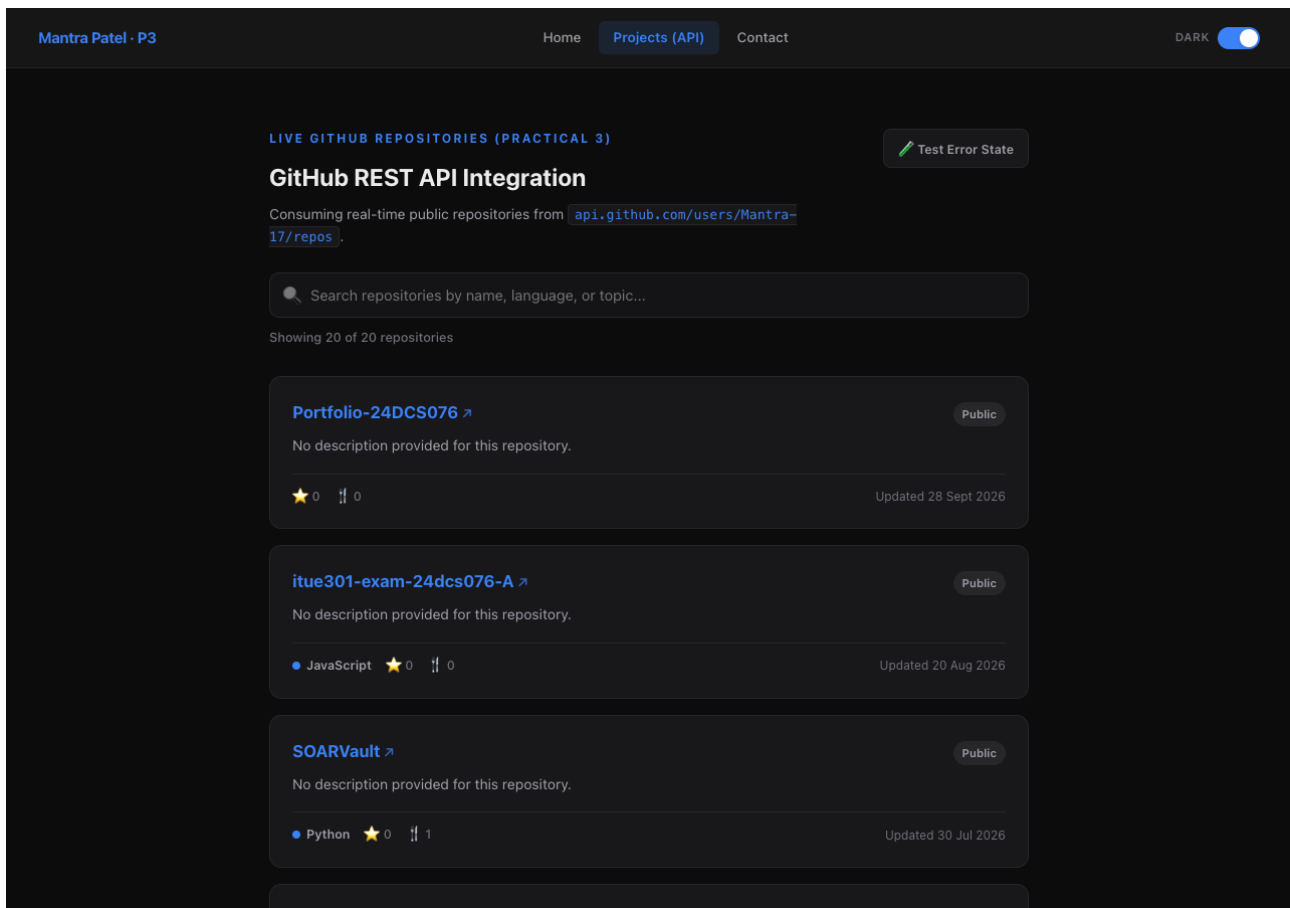


Figure: Practical 3 - After Retry: repositories successfully reloaded after error recovery

5. Testing Checklist

- npm install runs without errors in Practical-3/
- npm run dev launches on `http://localhost:5175` with `strictPort: true`
- GitHub REST API fetches real repos for Mantra-17 on first mount
- Loading spinner renders visibly while request is in-flight
- Success state displays all repository cards with name, URL, stars, language
- Real-time search input instantly filters repo list on every keystroke
- Clearing the search restores the full repository list
- Typing a non-matching term shows the empty state card (not a blank page)
- Error demo toggle triggers the error state without crashing the app
- Retry button fires `loadRepositories()` again and successfully restores repo data
- `useEffect` only runs once on mount (no infinite API request loop)
- Browser console: 0 errors, 0 warnings across all three states

6. Learning Outcomes

After completing Practical 3, I can:

- Consume a public REST API using the browser Fetch API inside a React component lifecycle.
- Use `useEffect` with a stable dependency (`useCallback`) to fetch data once on component mount without infinite loops.

- Model asynchronous request state as a three-value machine: loading / error / success.
- Implement a try/catch/finally block to correctly transition all three states on every fetch attempt.
- Compose specialised, reusable components (Spinner, ErrorMessage, RepoCard, RepoList) from a single data-fetching orchestrator (Projects.jsx).
- Separate HTTP concerns into a standalone service module (githubApi.js).
- Implement client-side search filtering using `Array.filter()` and `String.includes()`.
- Handle edge cases: HTTP 404/403 status codes, empty result sets, and network failures gracefully.